Image Processing & Computer Vision

A 3-day PyTorch course

Markus Hinsche · datascienceretreat.com · 2026

Agenda

Over three days we'll cover:

- **Day 1** — images as tensors, convolutions, datasets & augmentations
- **Day 2** — building and training CNNs, transfer learning, Grad-CAM
- **Day 3** — attention, building a ViT from scratch, CNN vs ViT

Hands-on throughout. Every concept is paired with a notebook you run yourself.

Topics we can dig into on request

- image augmentation strategies (RandAugment, MixUp, CutMix, Albumentations)
- self-supervised pretraining (SimCLR, DINO, MAE)
- object detection (YOLO, DETR), instance/semantic segmentation (U-Net, Mask R-CNN, SAM), pose estimation, keypoint detection
- vision-language models (CLIP, SigLIP, BLIP) and zero-shot classification
- robustness: adversarial examples, distribution shift, OOD detection
- active learning and data-centric labeling loops for image datasets
- medical imaging (+ class imbalance)
- synthetic data and sim-to-real

Download slides and code

Clone the repo to follow along:

```
$ git clone https://github.com/markus-hinsche/img-proc-and-comp-vision.git
$ cd img-proc-and-comp-vision
$ uv sync
$ uv run jupyter lab
```

Notebooks live in `notebooks/`, slides in `slides/`.

whoami

- **Markus Hinsche** — Co-Founder & CTO of Thea Care
  - B2B SaaS: AI skin analysis for cosmetic and pharma brands
  - Computer vision in production (segmentation + classification)
- Previously: ML freelancer — Welthungerhilfe, Charité radiology (both CV)
- Strengths:
  - Startups
  - Python (since 2009), Deep Learning (since 2018)
  - Productionizing models, MLOps, mentoring
- markushinsche.de · github

Why computer vision?

"Vision is the killer app of deep learning."

Image classification was the first task where deep nets clearly beat classical methods (AlexNet, 2012). The recipes you learn this week — convolutions, transfer learning, attention — are the foundation for everything from medical imaging to robotics to generative models.

What you'll be able to do by the end of the class

- Load any image dataset into PyTorch and apply augmentations
- Build a CNN from scratch and train it on GPU
- Fine-tune a pretrained model (ResNet, EfficientNet, ViT) on your own data
- Debug training failures (loss not decreasing, overfitting, wrong shapes)
- Explain *why* a model predicts what it predicts (Grad-CAM)
- Understand the bridge from CNNs to Vision Transformers

Ground rules

- **Ask questions immediately.** If you're lost on slide 3, you'll be lost on slide 30.
- **Modify the code.** Reading notebooks is not the same as running your new version.
- **Pair up.** Two brains debug faster than one.
- **It's OK if your model doesn't train.** Half the skill is figuring out why.

Building models in PyTorch

Two ways to define a model:

Approach	When to use it
`nn.Sequential(...)`	Straight-line stack, no branching. Tiny demos.
`nn.Module` subclass	The default — define layers, wire them in `forward`. Branches, skips, multiple heads, anything.

Plus **`torch.nn.functional`** (`F.relu`, `F.conv2d`, …) — stateless ops you call directly inside `forward()`. No layer object needed.

The two patterns side by side

```
# 1. nn.Sequential — only for linear stacks
model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(), nn.Linear(16*14*14, 10),
)

# 2. nn.Module subclass — the workhorse. Branches, skips, anything.
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 16, 3, padding=1)
        self.head = nn.Linear(16*14*14, 10)
    def forward(self, x):
        x = F.relu(self.conv(x))
        x = F.max_pool2d(x, 2)
        return self.head(x.flatten(1))
```

Rule of thumb: start with `nn.Sequential`, switch to a subclass the moment you need a skip connection, two heads, or any conditional flow.

Debugging CV code

Failure modes you'll meet first — a reference deck. Point back to it any time you hit a bug. The goal: make *"what kind of bug is this?"* a reflex.

The five bugs that eat the most time (1/2)

1. Wrong tensor shape

Symptom: `RuntimeError: mat1 and mat2 shapes cannot be multiplied` or "expected input of shape (N, 3, H, W) but got (N, H, W, 3)".

Reflex: print `.shape` at every step. Especially before and after `Flatten`, `permute`, `view`, and the first `Linear`.

2. HWC vs CHW

Symptom: image looks like static, or accuracy stuck at random.

Cause: loaded an image as `(H, W, C)` and fed it to a Conv2d expecting `(C, H, W)`.

Reflex: if shape `[..., 3]` ends in a small number, you probably have HWC.

3. Forgot `.to(device)`

Symptom: `Expected all tensors to be on the same device, but found at least two devices, cpu and cuda:0`.

The five bugs that eat the most time (2/2)

4. Forgot `optimizer.zero_grad()`

Symptom: loss curve looks bizarre, gradients explode.
Cause: PyTorch *accumulates* gradients. Without `zero_grad()`, each step uses the sum of all previous gradients.
Reflex: the loop pattern is fixed — don't reorder:

```
optimizer.zero_grad(); loss.backward(); optimizer.step()
```

5. `train()` vs `eval()` mode

Symptom: validation accuracy randomly bounces; dropout/batchnorm misbehave.
Cause: stayed in `model.train()` during evaluation, or never switched back.
Reflex: any eval/inference block starts with `model.eval()` and `with torch.no_grad():`.

Three more, less common but nasty

Normalization mismatch

If you fine-tune a torchvision model, you must use ImageNet mean/std. If you trained from scratch on your dataset, use *its* mean/std. Mixing them silently degrades accuracy.

Augmenting validation

Augmentations belong only on the train set. Augmenting validation makes the metric noisy and uncomparable.

Wrong `Flatten` size

`nn.Linear(in_features, ...)` must match `C * H * W` after the last conv block. Compute it on paper from the shape rule, or run one batch through and read the shape off `Flatten` output.

The 30-second sanity check

Before training anything:

```
xb, yb = next(iter(train_loader))
xb, yb = xb.to(device), yb.to(device)
logits = model(xb)
print(xb.shape, "->", logits.shape)
print("loss on init:", F.cross_entropy(logits, yb).item())
```

If this runs without error and the loss is roughly `-log(1/num_classes)`,
you're set to train.

Let's go.

Open `notebooks/01_images_as_tensors.ipynb`